



life.augmented



STM32WL55

Hardware semaphores (HSEM) in practice

T.O.M.A.S. Team

Goals of the session

Using STM32CubeIDE, I will demonstrate how to:

- Assigning peripherals to both cores (i.e USART2, HSEM)
- Select, configure and use hardware semaphores (HSEM)
- Monitor peripherals within debug session on two cores
- Use built in terminal within STM32CubeIDE

Prerequisites

- NUCLEO-WL55JC2 board
- Micro USB cable
- PC with preinstalled the following software:
 - STM32CubeIDE (in version 1.5.0 or newer)
 - STM32WL Cube library (in version 1.0.0 or newer)

Hardware semaphores (HSEM) theory



Hardware semaphores (HSEM) in STM32WL55

main points

- Within dual core STM32WL55 MCU there are 16 hardware semaphores.
- Each semaphore stores an information about the core and process which took it (lock it)
- If the semaphore is not taken, its status registers are cleared (HSEM_RLRx and HSEM_Rx).
- Semaphores can be used within single code or between cores to synchronize the processes
- A semaphore can be locked when it is free and PROCID field is cleared within its configuration
- After the reset semaphores are enabled for both cores but access to them can be separately disabled for each of the core
- There is single interrupt for HSEM which can be triggered by selected semaphores on unlock event.

Hardware semaphores (HSEM) in STM32WL55

extract from Reference Manual (RM0453)

- Each of hardware semaphore has special registers and read lock registers which contain:
 - LOCK**: 1 status bit (0 – semaphore free, 1 – semaphore taken/locked)
 - COREID**: 4 bit identifier of the core which took the semaphore:
 - 4 for ARM CortexM4
 - 8 for ARM CortexM0+
 - 0 for free semaphore
 - PROCID**: 8 bit identifier of the process which lock the semaphore (0..0xFF value)
- When the semaphore is free all above fields are cleared

HSEM register semaphore x (HSEM_Rx)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LOCK	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
rw															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	COREID[3:0]				PROCID[7:0]							
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

read/write access

HSEM read lock register semaphore x (HSEM_RLRx)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LOCK	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	COREID[3:0]				PROCID[7:0]							
				r	r	r	r	r	r	r	r	r	r	r	r

read only access

Hardware semaphores (HSEM) in STM32WL55 extract from Reference Manual (RM0453)

- It is possible to release at once all semaphores assigned to selected core
- To do this we need to specify 16bit key value within HSEM_KEYR (common for both cores)
- Then if we write the proper key to HSEM_CR register (bits.16..31) (the same like specified in HSEM_KEYR), all semaphores locked by the core which was accessing HSEM_CR will be released.

HSEM clear register (HSEM_CR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	COREID[3:0]				Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
				w	w	w	w								

write only access

HSEM interrupt clear register (HSEM_KEYR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEY[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

read/write access

Hardware semaphores (HSEM) API

stm32wlxx_hal_hsem.c

- Within STM32WL Cube HAL library there is a following set of functions supporting HSEM
 - `HAL_StatusTypeDef HAL_HSEM_Take(uint32_t SemID, uint32_t ProcessID)`
 - `HAL_StatusTypeDef HAL_HSEM_FastTake(uint32_t SemID)`
 - `uint32_t HAL_HSEM_IsSemTaken(uint32_t SemID)`
 - `void HAL_HSEM_Release(uint32_t SemID, uint32_t ProcessID)`
 - `void HAL_HSEM_ReleaseAll(uint32_t Key, uint32_t CoreID)`
 - `void HAL_HSEM_SetClearKey(uint32_t Key)`
 - `uint32_t HAL_HSEM_GetClearKey(void)`
 - `void HAL_HSEM_ActivateNotification(uint32_t SemMask)`
 - `void HAL_HSEM_DeactivateNotification(uint32_t SemMask)`
 - `void HAL_HSEM_IRQHandler(void)`
 - `__weak void HAL_HSEM_FreeCallback(uint32_t SemMask)`

SemID 0..31

ProcessID 0..255

CoreID:

4 for CM4

8 for CM0+

Key 0..0xFFFF

SetMask mask of semaphores ID 0..0xFFFF

Hardware semaphores (HSEM) in STM32WL55

how to take / lock the semaphore

There are two ways to lock the semaphore:

- 2-step lock procedure (in HAL **HAL_HSEM_Take()** function)
 - Write semaphore with PROCID and COREID, and LOCK = 1. The COREID data written by software must match the AHB bus master information. i.e. a AHB bus master ID = 1 writes data COREID = 1. Lock is put in place when the semaphore is free at write time.
 - Read-back the semaphore The software checks the lock status, if PROCID and COREID match the written data, then the lock is confirmed.
 - Else retry (the semaphore has been locked by another process, AHB bus master ID)
- 1-step lock procedure (in HAL **HAL_HSEM_FastTake()** function)
 - Read lock semaphore with the AHB bus master COREID.
 - If read COREID matches and PROCID = 0, then lock is put in place. If COREID matches and PROCID is not 0, this means that another process from the same COREID has locked the semaphore with a 2-step (write) procedure.
 - Else retry (the semaphore has been locked by another process, AHB bus master ID).

Hardware semaphores (HSEM) in STM32WL55

how to unlock the semaphore

There are two ways to unlock semaphore(s):

- Write HSEM_Rx register with PROCID, COREID, and LOCK = 0 (in HAL **HAL_HSEM_Release()** function)
- Write HSEM_CR register with previously set Key and specify COREID to unlock all semaphores locked/taken by selected COREID (in HAL **HAL_HSEM_ReleaseALL()** function)

After hardware semaphore is unlocked, an interrupt may be generated if:

- HSEM interrupts are configured and enabled
- Particular hardware semaphore is selected as possible interrupt trigger (in HAL **HAL_HSEM_ActivateNotification** function)

Thank you

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.



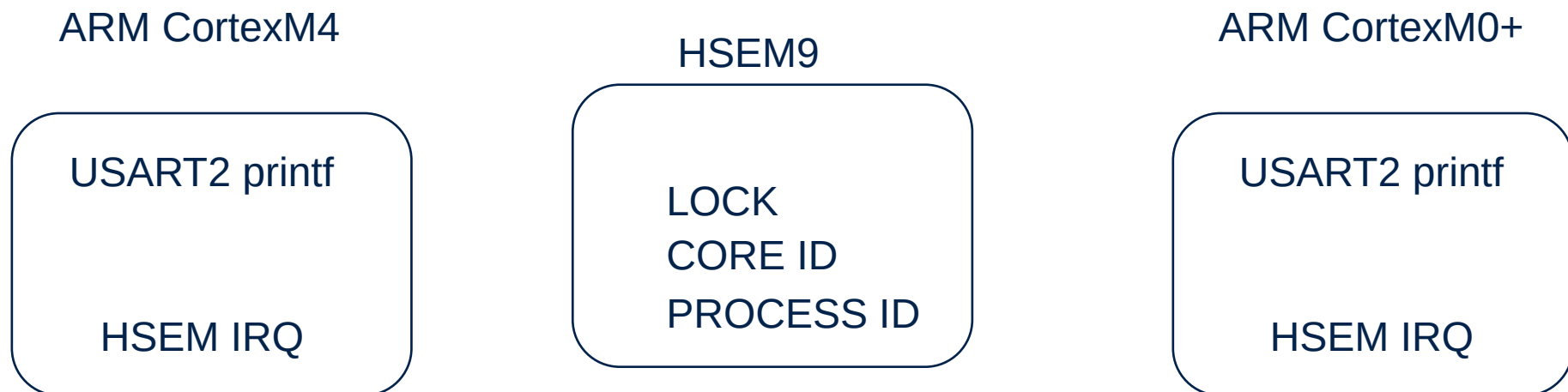
life.augmented

Hardware semaphores (HSEM) example



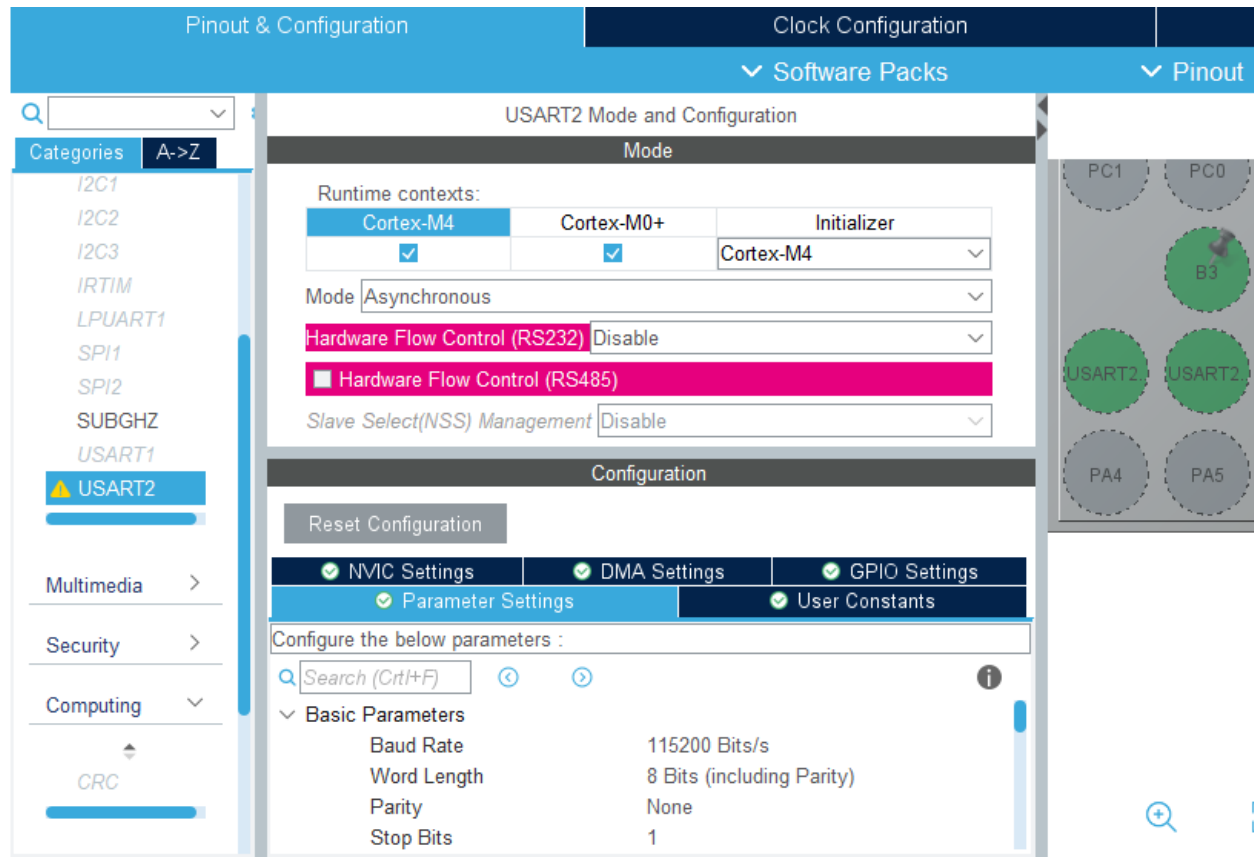
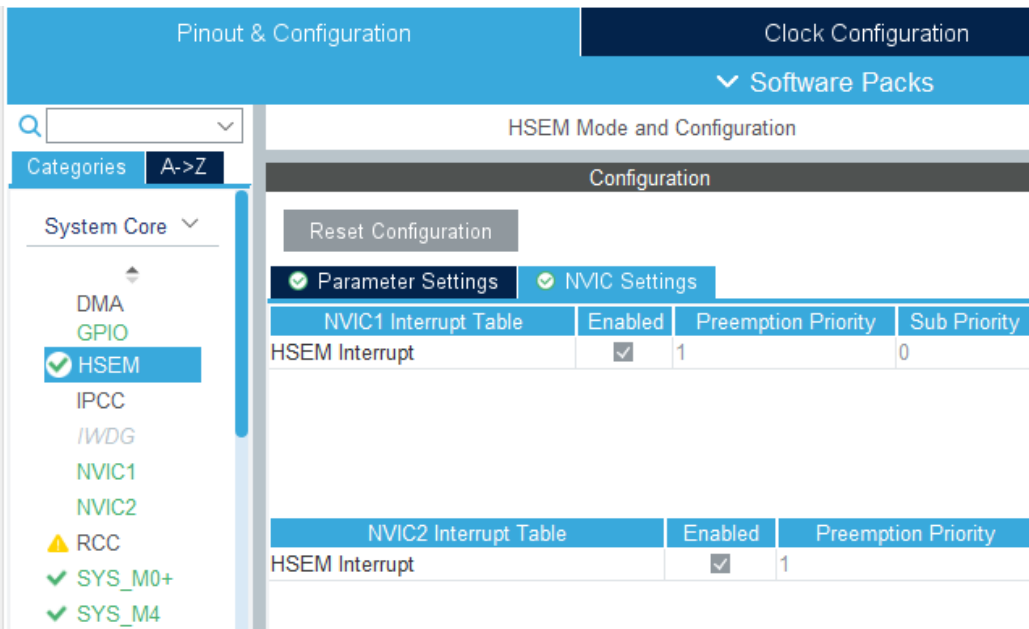
Hardware semaphores (HSEM) example description

- We can reuse previous example (with 3 buttons and 3 LEDs) or create a new one
- We will add USART2 and HSEM and assign them to both cores
- USART2 will be configured within CortexM4 core
- Additionally in main while(1) loop of both cores we will perform simple cyclic data send over USART2
- Access to USART2 will be protected by selected hardware semaphore
- We will use HSEM interrupts to inform about semaphore release.



Hardware semaphores (HSEM) example configuration part

- We may reuse previous example (with 3 buttons and 3 LEDs) or create a new project for our MCU
- Additionally:
 - we will enable UART2 (PA2, PA3) to be used by both cores (configured within CM4)
 - for USART2 we will set 115200bps / 8 b data / 1 stop bit / no parity / no interrupts / no DMA
 - we will enable HSEM for both cores
 - we will enable HSEM interrupt generation on both cores as well



Hardware semaphores (HSEM) example coding part – ARM CortexM4 main.c file

- Specify which HW semaphore will be used and what will be the process number

```
/* USER CODE BEGIN PD */  
#define HSEM_ID (9U) /* use HW semaphore 9 */  
#define HSEM_PROCESS 4U
```

- Declare variable to be used within HSEM interrupt to signal semaphore release

```
/* USER CODE BEGIN PV */  
uint8_t semaphore_status = 0;
```

- Activate interrupt notifications on selected semaphore (semaphore 9 in this example)

```
/* USER CODE BEGIN 2 */  
HAL_HSEM_ActivateNotification(__HAL_HSEM_SEMID_TO_MASK(HSEM_ID));
```



Hardware semaphores (HSEM) example coding part – ARM CortexM4 main.c file

- Within main while() loop try to take the semaphore to send the message over USART2 then release the semaphore (using the same process number)

```
/* USER CODE BEGIN WHILE */
while (1)
{
    if(HAL_HSEM_Take(HSEM_ID, HSEM_PROCESS) == HAL_OK)
    {
        semaphore_status=0;
        HAL_NVIC_DisableIRQ(HSEM_IRQn);
        HAL_UART_Transmit(&huart2, "CM4\r\n", 6, 200);
        HAL_HSEM_Release(HSEM_ID, HSEM_PROCESS);
    }
    else
    {
        HAL_NVIC_EnableIRQ(HSEM_IRQn);
        while(semaphore_status == 0); //wait till HSEM interrupt will come
    }
    HAL_Delay(1000); //just for better visibility on terminal
}
```

Hardware semaphores (HSEM) example coding part – ARM CortexM4 main.c file

- Within HSEM interrupt callback set semaphore_status to 1 to inform about semaphore release

```
void HAL_HSEM_FreeCallback(uint32_t SemMask)
{
    if((SemMask & __HAL_HSEM_SEMID_TO_MASK(HSEM_ID))!= 0)
    {
        semaphore_status = 1;
    }
}
/* USER CODE END 4 */
```

Hardware semaphores (HSEM) example coding part – ARM CortexM0+ main.c file

- Specify which HW semaphore will be used and what will be the process number

```
/* USER CODE BEGIN PD */
#define HSEM_ID (9U)      /* use HW semaphore 9 - must be the same like for CortexM4 */
#define HSEM_PROCESS 4U  /* can be different than for CortexM4 code */
```

- Declare variable to be used within HSEM interrupt to signal semaphore release

```
/* USER CODE BEGIN PV */
uint8_t semaphore_status = 0;
```

- Activate interrupt notifications on selected semaphore (semaphore 9 in this example)

```
/* USER CODE BEGIN 2 */
HAL_HSEM_ActivateNotification(__HAL_HSEM_SEMID_TO_MASK(HSEM_ID));
/* next lines are necessary to set huart2 initial data as USART2 initialization
   has been done by CM4 */
huart2.Instance = USART2; //basic data for the handler to use with proper peripheral
UART_CheckIdleState(&huart2); //set TX and RX to non busy initial state
```

Hardware semaphores (HSEM) example coding part – ARM CortexM0+ main.c file

- Within main while() loop try to take the semaphore to send the message over USART2 then release the semaphore (using the same process number)

```
/* USER CODE BEGIN WHILE */
while (1)
{
    if(HAL_HSEM_Take(HSEM_ID, HSEM_PROCESS) == HAL_OK)
    {
        semaphore_status=0;
        HAL_NVIC_DisableIRQ(HSEM_IRQn);
        HAL_UART_Transmit(&huart2, "CM0+\r\n", 7, 200);
        HAL_HSEM_Release(HSEM_ID, HSEM_PROCESS);
    }
    else
    {
        HAL_NVIC_EnableIRQ(HSEM_IRQn);
        while(semaphore_status == 0);
    }
    HAL_Delay(1000);
}
```

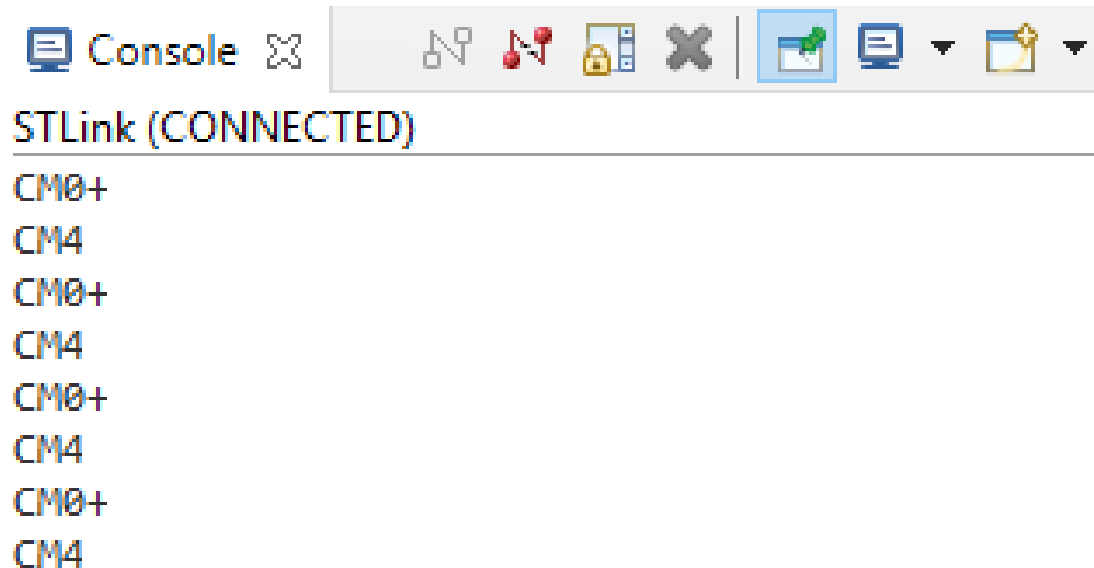
Hardware semaphores (HSEM) example coding part – ARM CortexM0+ main.c file

- Within HSEM interrupt callback set semaphore_status to 1 to inform about semaphore release

```
void HAL_HSEM_FreeCallback(uint32_t SemMask)
{
    if((SemMask & __HAL_HSEM_SEMID_TO_MASK(HSEM_ID)) != 0)
    {
        semaphore_status = 1;
    }
}
/* USER CODE END 4 */
```

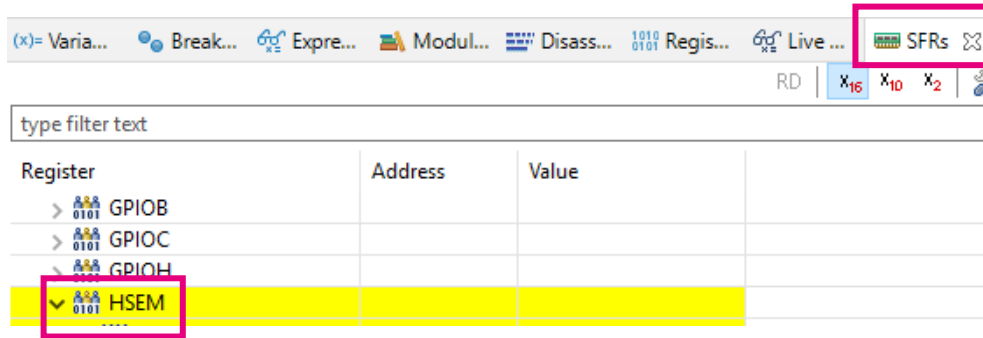
Hardware semaphores (HSEM) example start debug session

- Start CortexM4 debug configuration, then resume code execution (to start CortexM0+)
- Start CortexM0P debug configuration and resume code execution
- Open an external terminal application and select STLink VCP number, then set the same communication parameters like used in the code (115200bps, 8b data, 1 stop bit, no parity)
- Or open terminal window within STM32CubeIDE (more information on next slides)
- An example of the result is visible below



Hardware semaphores (HSEM) example debug part

- Once we stop on the breakpoint just after semaphore has been taken, we can check HSEM configuration looking at HSEM_R9 register (SFR tab in the right top corner during debug session)



▼ 1010 0101 HSEM_R9	0x58001424	0x80000404
1010 0101 PROCID	[0:8]	0x4
1010 0101 COREID	[8:4]	0x4
1010 0101 LOCK	[31:1]	0x1

ARM CortexM4 core
COREID 4

▼ 1010 0101 HSEM R9	0x58001424	0x80000804
1010 0101 PROCID	[0:8]	0x4
1010 0101 COREID	[8:4]	0x8
1010 0101 LOCK	[31:1]	0x1

ARM CortexM0+ core
COREID 8

Thank you

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.



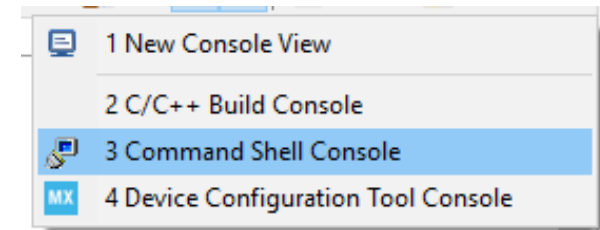
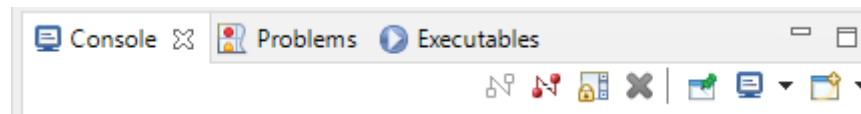
life.augmented

Built-in terminal in STM32CubeIDE

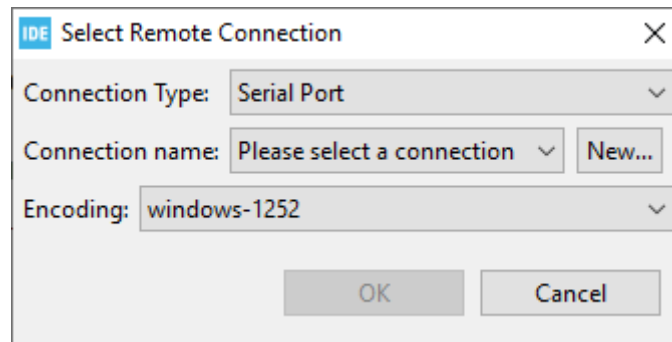


Configuring terminal within STM32CubeIDE

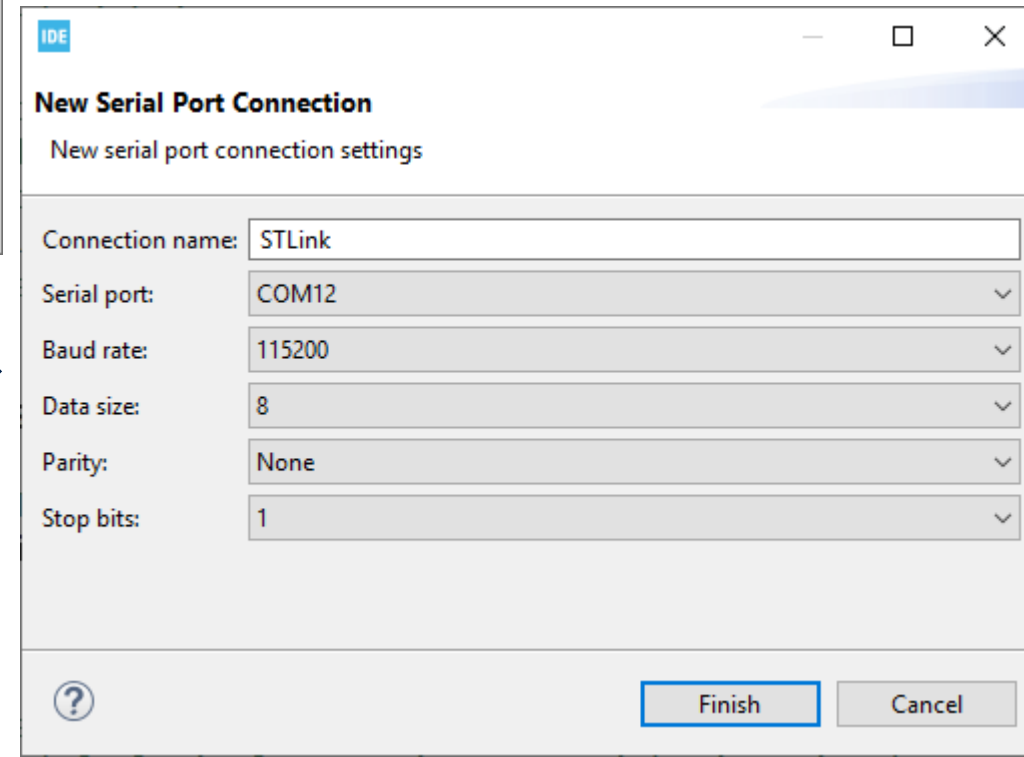
- Within debug session click on Open Console and select „3 Command Shell Console”



- Select „Serial Port” as connection type and choose „New...” for connection name



- Specify Connection name and its parameters and click on Finish



Using terminal within STM32CubeIDE

1. Select already configured console
2. Press on connect button (see below)
3. Resume code execution on both cores if needed

Terminal console menu main options:

-  Open console (start new terminal configuration)
-  Display selected console (start already configured one)
-  Connect
-  Disconnect
-  Close terminal console (remove active terminal configuration)

Thank you

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.



life.augmented



life.augmented



STM32WL55

Hardware semaphores (HSEM) in practice

T.O.M.A.S. Team

HSEM – Introduction and theory



HSEM – project configuration



HSEM – coding part



HSEM – debug session

